

Diseño de APIs

Basado en SE Radio 679: Wesley Beary on API Design

1. ¿Qué es una API?

Una API (Application Programming Interface) es un conjunto de reglas y protocolos que permite a distintos sistemas de software comunicarse entre sí. En términos prácticos, actúa como un intermediario que recibe peticiones de un cliente, las transmite a un servidor, y devuelve la respuesta correspondiente.

Las APIs se definen por tres características esenciales. En primer lugar, la abstracción: una API expone únicamente la funcionalidad necesaria sin revelar los detalles internos de su implementación. En segundo lugar, el contrato: la API establece con precisión cómo deben formularse las peticiones y cómo se estructurarán las respuestas, lo que genera confianza entre los equipos de desarrollo. Por último, la interoperabilidad: gracias a las APIs, sistemas contruidos con tecnologías completamente distintas pueden colaborar sin fricciones.

2. Las Suposiciones Incorrectas del Diseño (API)

El acrónimo API puede leerse de otra manera igualmente reveladora: "Assumptions Probably Incorrect" (Suposiciones Probablemente Incorrectas). Esta perspectiva recuerda una verdad fundamental del desarrollo de software: ningún plan de diseño sobrevive intacto al primer contacto con la realidad. Por muy cuidadoso que sea el análisis previo, el proceso de construcción y uso real de una API siempre revela decisiones que, en retrospectiva, podrían haberse tomado de manera diferente.

El flujo natural de este aprendizaje pasa por tres fases. Primero, el diseñador crea la API con las mejores intenciones y el conocimiento disponible. Segundo, al intentar usarla en un escenario real y complejo, afloran las incoherencias, los casos extremos no contemplados y las decisiones cuestionables. Tercero, se itera y corrige a la luz de esa experiencia.

3. Desarrollo Ágil Aplicado al Diseño de APIs

Wesley Beary propone aplicar los principios ágiles al diseño de APIs con una máxima clara: cometer los errores de la forma más barata posible. Para ello recomienda un flujo de trabajo en tres etapas que minimiza el coste de las correcciones y acelera el aprendizaje.

- **Spec:** antes de escribir una sola línea de código, se redacta el contrato de la API en formato OpenAPI (antes conocido como Swagger). Este documento describe los endpoints, los parámetros, los tipos de datos y los posibles errores. Al hacerlo desde el principio, el equipo se ve obligado a pensar con rigor en el diseño antes de comprometerse con una implementación.
- **Mocking:** existen herramientas que leen un spec de OpenAPI y generan automáticamente un servidor simulado con todos los endpoints

definidos. Esto permite al equipo de frontend o a los consumidores de la API empezar a integrar y probar sin necesidad de que el backend esté implementado.

- **Servidor:** cuando el diseño ya ha sido validado contra el mock y los casos de uso son bien conocidos, se implementa el servidor definitivo. En este punto, la probabilidad de cometer errores de diseño se ha reducido drásticamente, porque se parte de un conocimiento real del comportamiento.

4. Convertirse en Experto de APIs

Rara vez se diseña una API completamente desde cero. La mayoría de los diseñadores trabajan dentro de ecosistemas existentes, aprendiendo de las APIs que consumen a diario. Beary propone desarrollar una actitud crítica y reflexiva al interactuar con cualquier API, comparable a la de un experto en café que, mientras disfruta su taza, analiza el origen del grano, el perfil de tueste y el método de extracción.

La práctica consiste en formularse tres preguntas cada vez que se usa una API ajena: ¿qué me gusta de este diseño?, ¿qué me frustra?, y ¿por qué? Esta reflexión sistemática construye un criterio propio que, con el tiempo, se traduce en mejores decisiones de diseño.

5. El Principio de Mínima Sorpresa

El principio de mínima sorpresa, también llamado la Regla de Oro del diseño de APIs, establece que una API bien diseñada debe comportarse exactamente como su usuario espera. Si un desarrollador tiene que detenerse y leer la documentación para entender por qué un endpoint se comporta de una manera inesperada, hay un fallo de diseño. La sorpresa genera fricción, ralentiza la integración y aumenta el riesgo de errores.

Este principio se sustenta en dos pilares. La consistencia exige que patrones similares se resuelvan siempre de la misma manera a lo largo de toda la API: los mismos nombres para conceptos equivalentes, los mismos formatos de fecha, los mismos códigos de error para situaciones análogas. La descubribilidad implica que la estructura de la API sea tan intuitiva que el desarrollador pueda anticipar la existencia de un endpoint o el formato de un parámetro antes de haberlo buscado en la documentación.

6. CRUD vs. Acciones

El modelo CRUD (Create, Read, Update, Delete) es la base del diseño REST, pero en muchos casos resulta insuficiente. El problema surge cuando las operaciones del dominio de negocio no encajan limpiamente en ninguna de esas cuatro operaciones. ¿Cómo se modela, por ejemplo, la acción de "aprobar una solicitud" o "cancelar un pedido"? Forzar estas semánticas dentro de un PATCH o un PUT puede resultar confuso e inexpressivo.

El Patrón de Acciones ofrece una solución más semántica: se definen endpoints que representan explícitamente la acción de negocio, como POST

/pedidos/{id}/cancelar. La ventaja de este enfoque es doble: el código que consume la API es más legible y expresivo, y el diseño de la API refleja con mayor fidelidad el lenguaje del dominio, facilitando la colaboración entre equipos técnicos y de negocio.

7. Versionado: El Gran Dilema

El versionado de APIs es uno de los retos más debatidos en la comunidad. Cuando una API evoluciona y sus cambios son incompatibles con versiones anteriores, los consumidores existentes pueden verse afectados. La pregunta es: ¿cómo gestionar esa evolución de forma ordenada y sin romper integraciones?

Existen diversas estrategias: incluir la versión en la URL (como /v1/recursos), usar cabeceras HTTP personalizadas, o apostar por la compatibilidad hacia atrás evitando cambios rupturistas. Cada enfoque tiene sus ventajas e inconvenientes en términos de claridad, mantenimiento y experiencia del desarrollador. Lo importante es tomar una decisión consciente y aplicarla de forma consistente en toda la API.

8. Documentación y OpenAPI como Contrato

Uno de los problemas más comunes en proyectos a largo plazo es el drift o desviación: la documentación deja de reflejar el comportamiento real de la API porque la implementación evoluciona sin que la documentación se actualice. Esto genera desconfianza y obliga a los desarrolladores a leer el código fuente en lugar de confiar en el contrato definido.

La solución propuesta por Beary es tratar el spec de OpenAPI no como una documentación secundaria, sino como el contrato central del proyecto. Si el spec se escribe primero y se mantiene como fuente de verdad, el drift se previene desde el origen. Además, este enfoque habilita el mocking automático: herramientas como Prism o Mockoon pueden generar servidores simulados a partir del spec, lo que permite al equipo de clientes trabajar en paralelo al equipo de backend desde el primer día. Tal como resume Beary: escribe el spec primero, luego escribe el cliente contra un mock, y deja la implementación del servidor para el final, cuando ya tienes toda la información y es menos probable que cometas errores.